

Welcome to CS101B!

Who's Here Today?

- Aeronautics and Astronautics
- Bioengineering
- Biology
- Chemical Engineering
- Chemistry
- Civil / Environmental Engineering
- Computational and Mathematical Engineering
- Computer Science
- Creative Writing
- Design
- Earth Planetary Sciences
- Earth Systems
- Economics
- Education
- Electrical Engineering
- Engineering
- Ethics in Society
- History
- Human Biology
- Immunology
- Law
- Mechanical Engineering
- Medicine
- Management Science and Engineering
- Physics
- Political Science
- Spanish
- Statistics
- Stem Cell Engineering / Regenerative Medicine
- Symbolic Systems
- Theater and Performance Studies
- ***Undeclared!***

Prerequisites

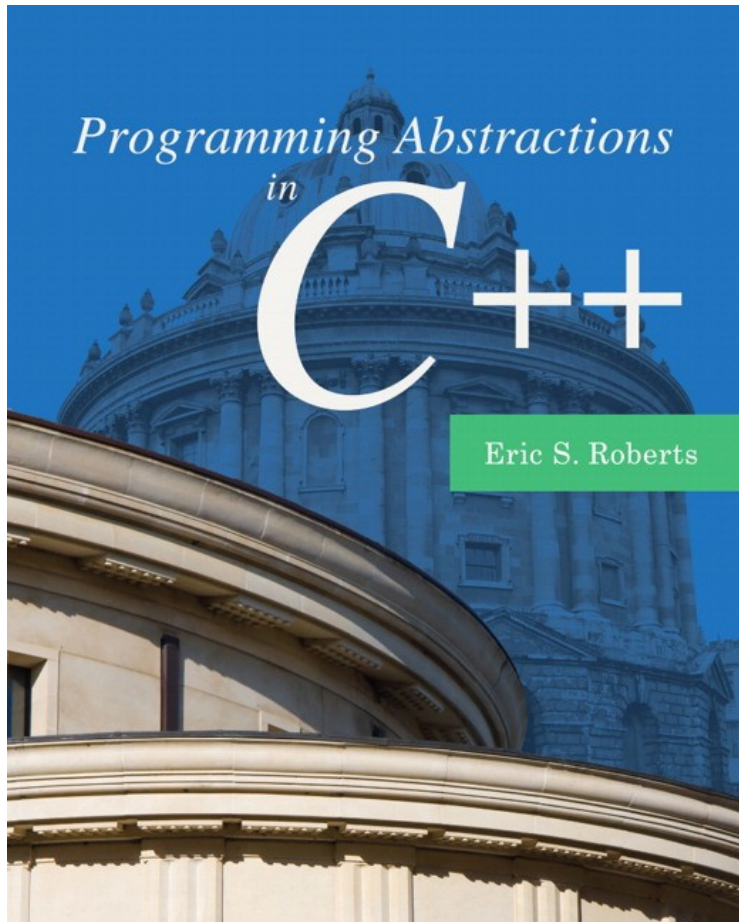
CS101A

(or equivalent)

60-Minute Lectures

- We have an 80-minute time slot for lectures this quarter, but we'll only use 60 of those minutes.
- Compared with a traditional 50-minute lecture, those extra ten minutes give us time to
 - answer your questions,
 - explore and tinker with code,
 - go at a more leisurely pace, and
 - let you play around with the material.
- I'll stick around for the remaining 20 minutes of our time block to chat with folks one-on-one about whatever it is that you're interested in.

Our Textbook



- Our course textbook is ***Programming Abstractions in C++*** by the legendary Eric Roberts.
 - There's a **draft version** available online.
- We've assigned readings for each lecture. You can either do them before or after the lectures – your choice.
- It's important to complete the readings in addition to attending lecture; there's a lot of really good info in there.

Grading Policies

Grading Policies



■ 40% Assignments

**Eight Coding
Assignments**

Plus an intro assignment
that goes out today and is
due Friday.

Grading Policies

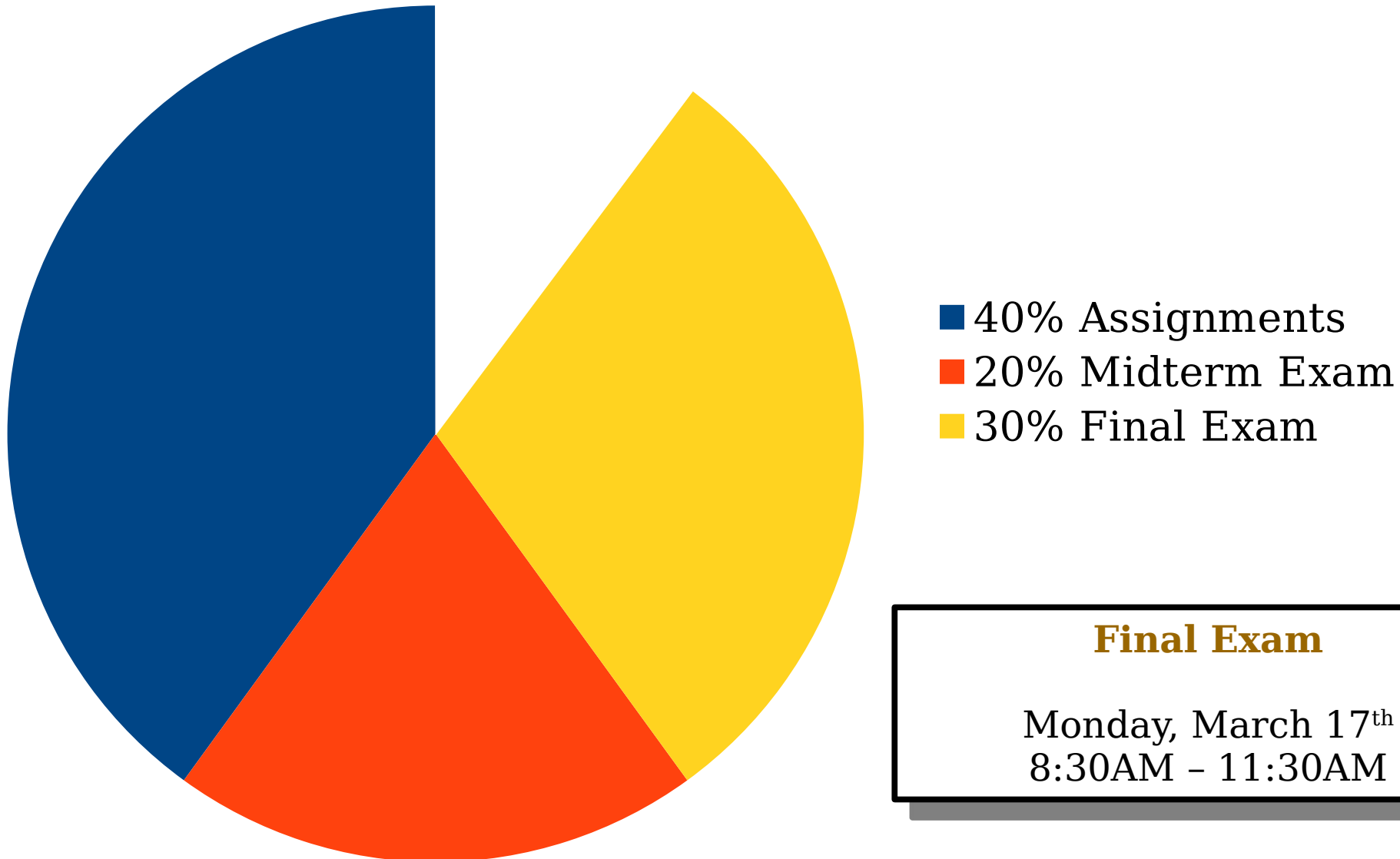


- 40% Assignments
- 20% Midterm Exam

Midterm Exam

Monday, February 10th
7PM - 10PM

Grading Policies



Grading Policies

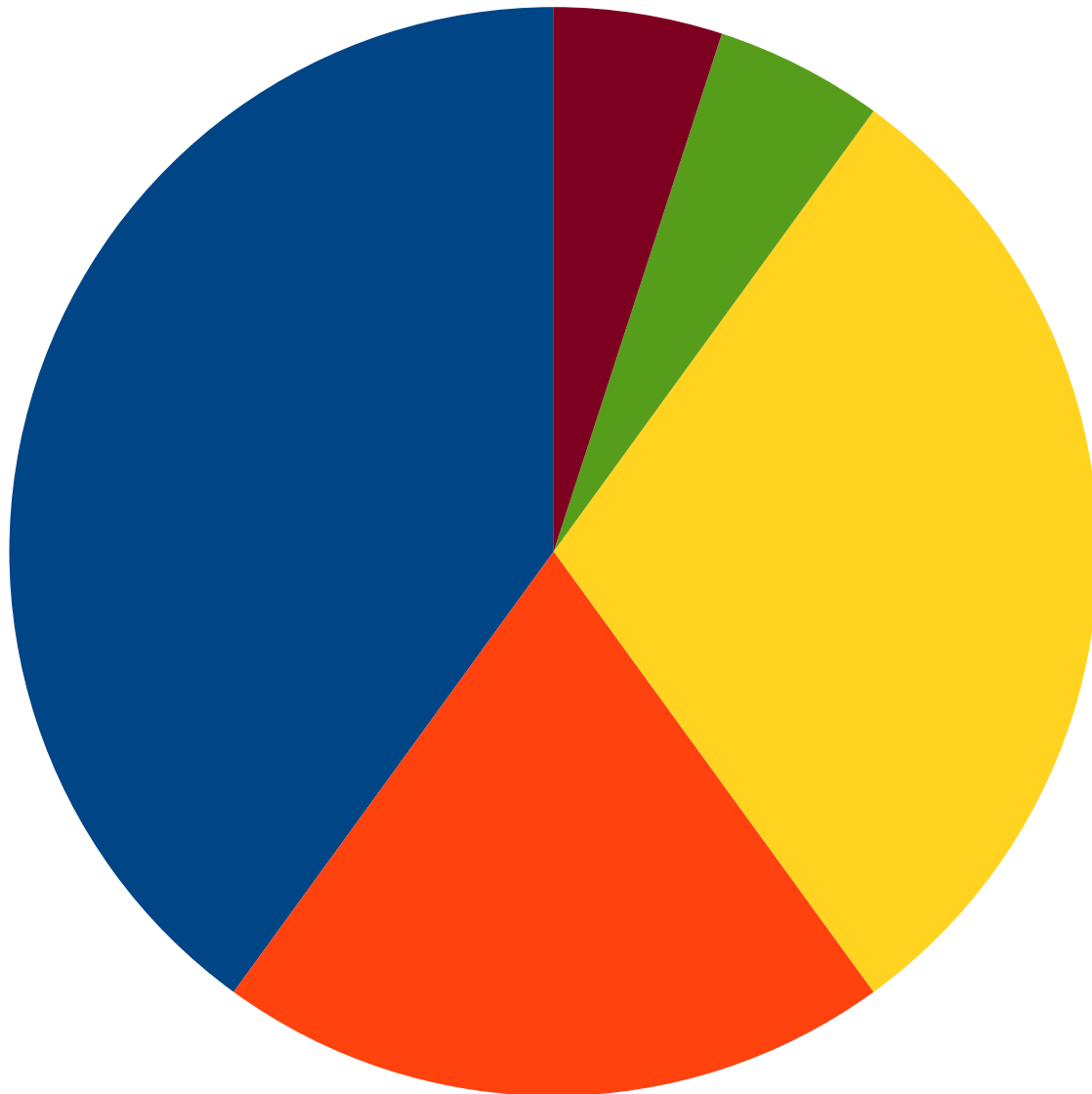


- 40% Assignments
- 20% Midterm Exam
- 30% Final Exam
- 5% Section Participation

Discussion Sections

Our world-famous
discussion sections!

Grading Policies



- 40% Assignments
- 20% Midterm Exam
- 30% Final Exam
- 5% Section Participation
- 5% Lecture Participation

Lecture Participation

Starts next week. We'll discuss details later this week.

What's Next in Computer Science?

Goals for this Course

- ***Learn how to model and solve complex problems with computers.***
- To that end:
 - Explore common abstractions for representing problems.
 - Harness recursion and understand how to think about problems recursively.
 - Quantitatively analyze different approaches for solving problems.

Goals for this Course

Learn how to model and solve complex problems with computers.

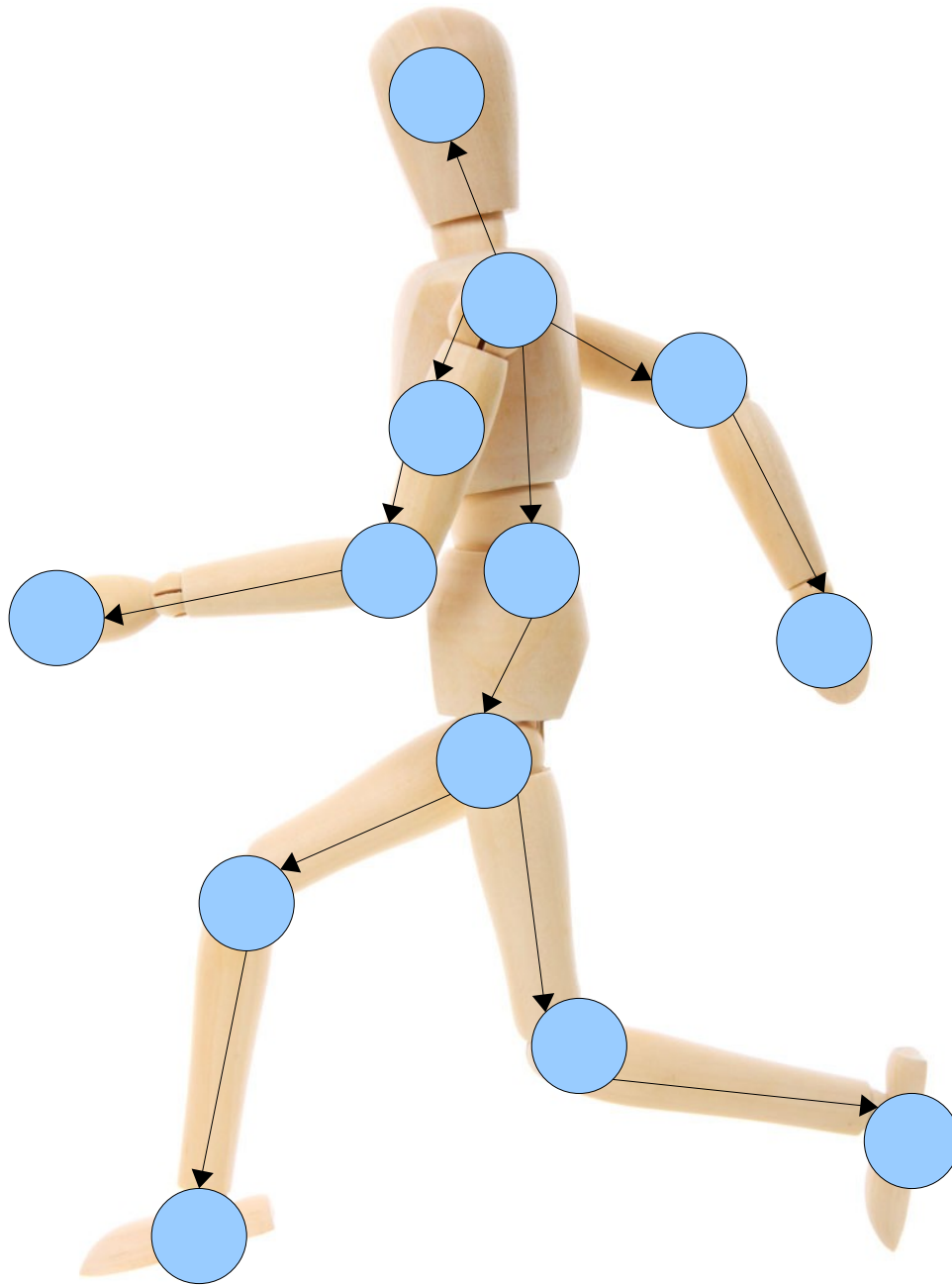
To that end:

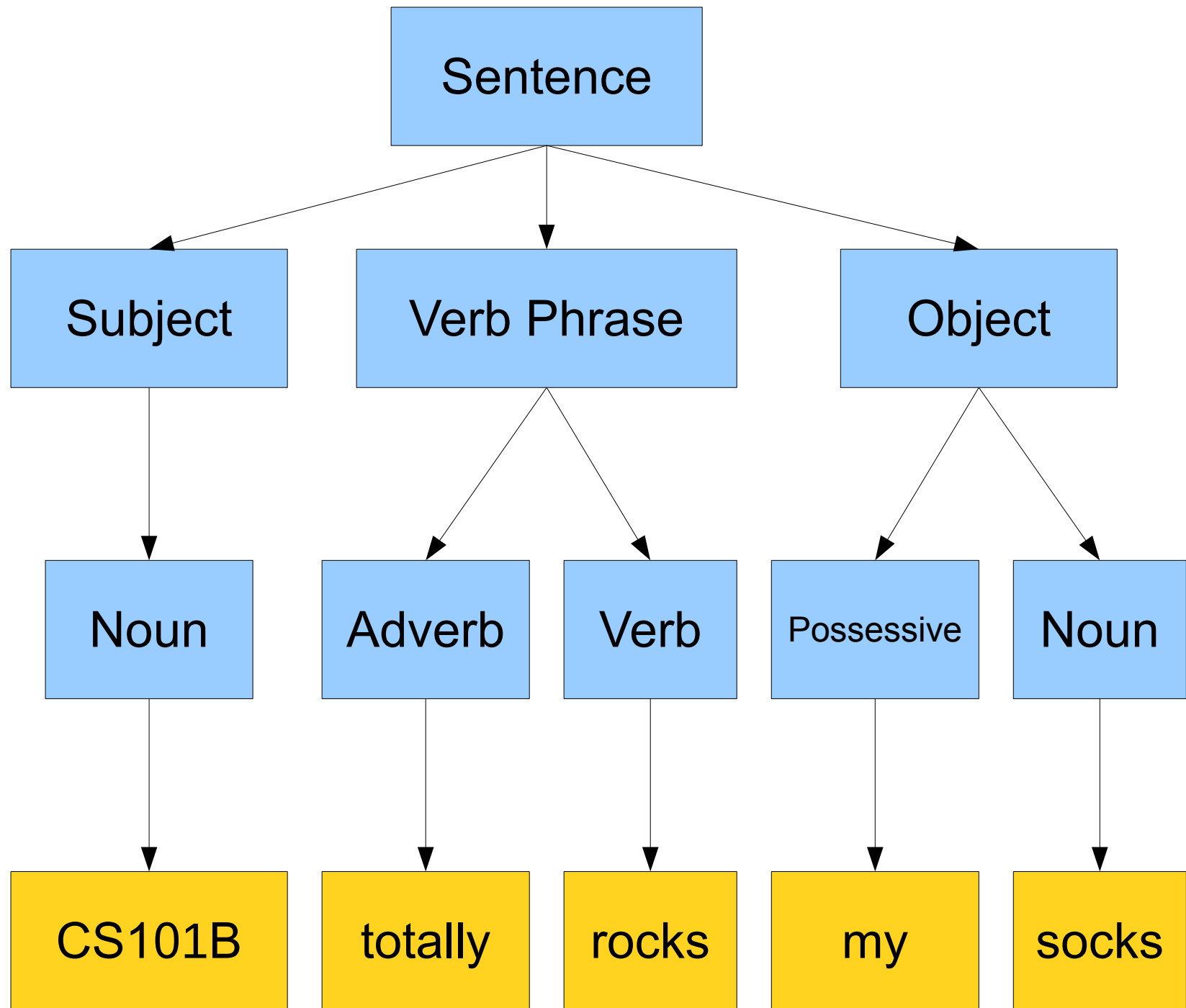
- Explore common abstractions for representing problems.

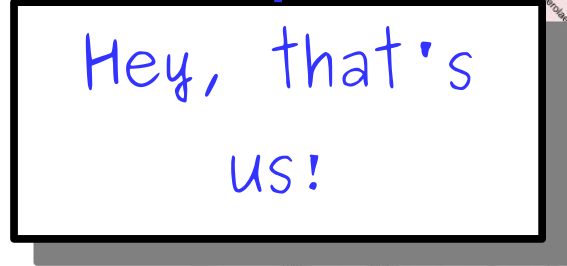
Harness recursion and understand how to think about problems recursively.

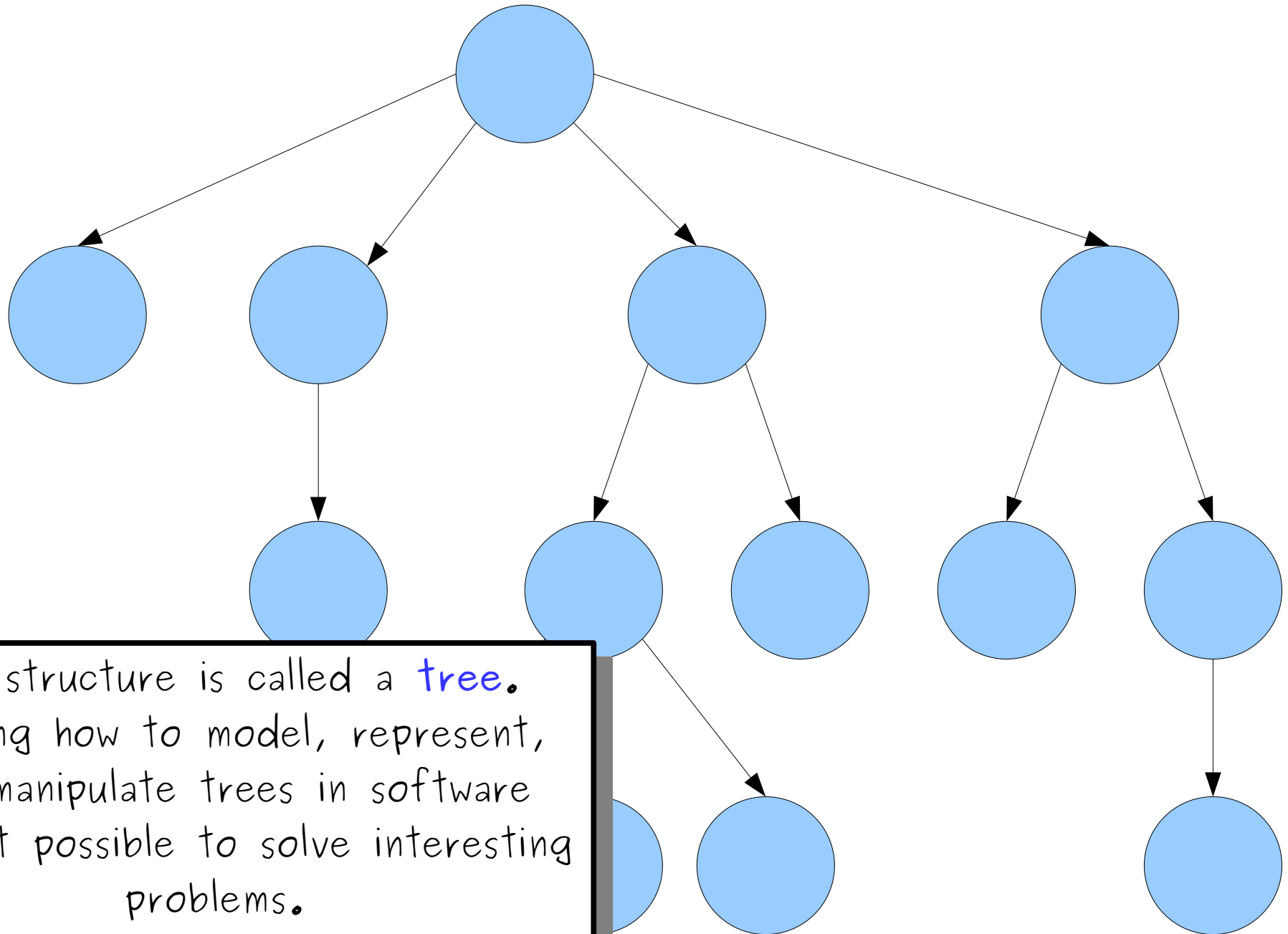
Quantitatively analyze different approaches for solving problems.











This structure is called a **tree**.
Knowing how to model, represent,
and manipulate trees in software
makes it possible to solve interesting
problems.

Building a vocabulary of ***abstractions*** makes it possible to represent and solve a wider class of problems.

Goals for this Course

- ***Learn how to model and solve complex problems with computers.***
- To that end:
 - Explore common abstractions for representing problems.
 - Harness recursion and understand how to think about problems recursively.
 - Quantitatively analyze different approaches for solving problems.

Goals for this Course

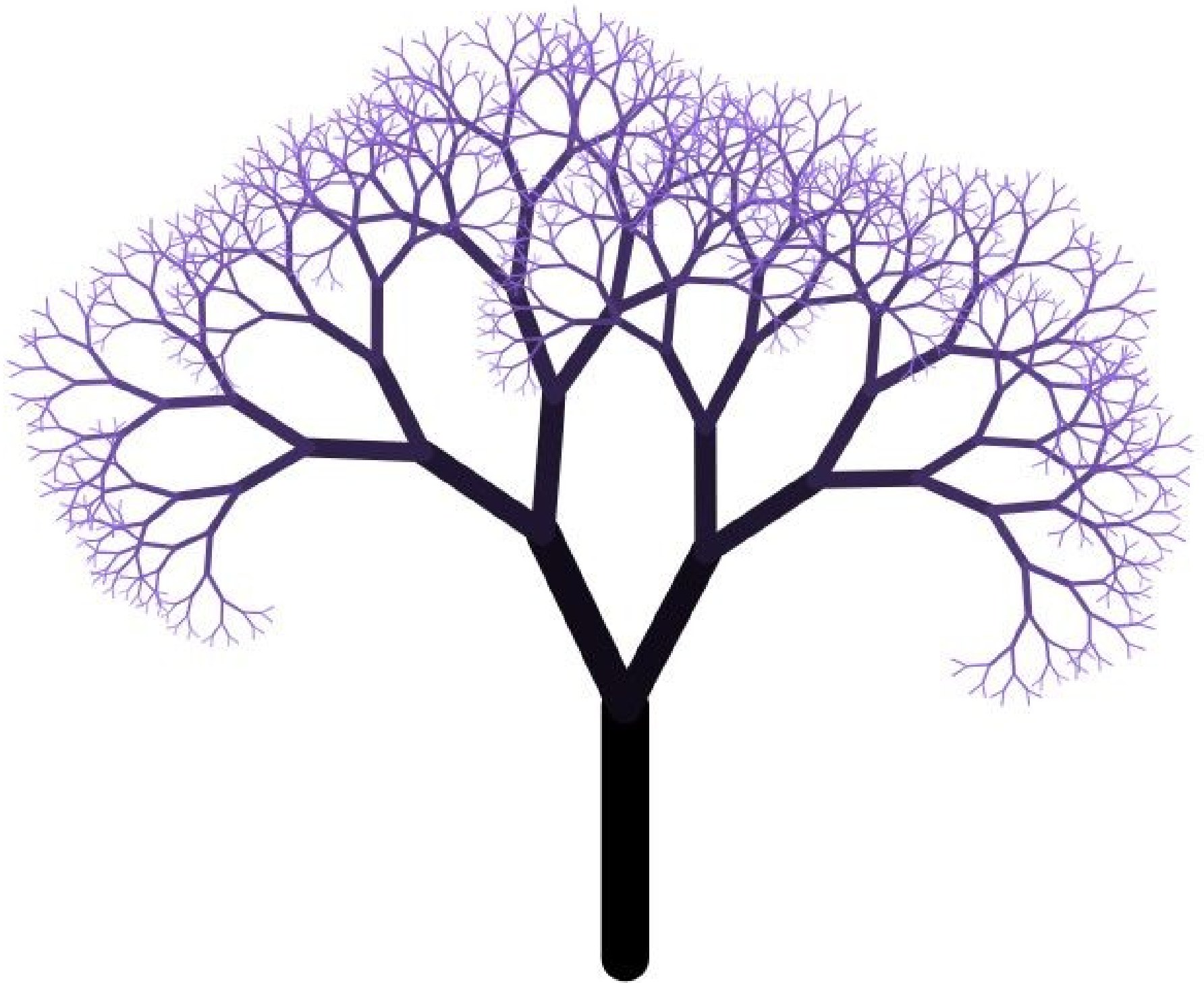
Learn how to model and solve complex problems with computers.

To that end:

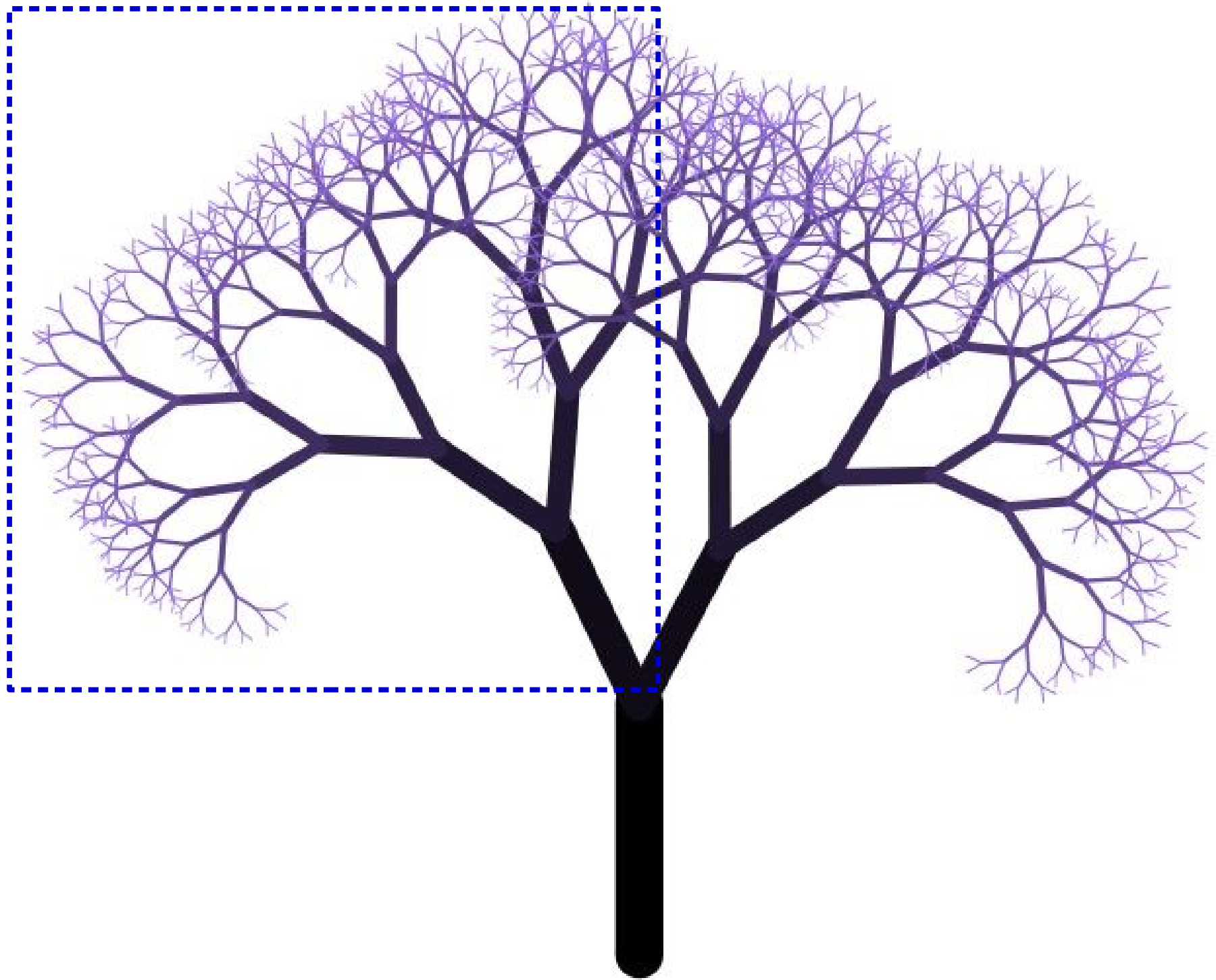
Explore common abstractions for representing problems.

- Harness recursion and understand how to think about problems recursively.

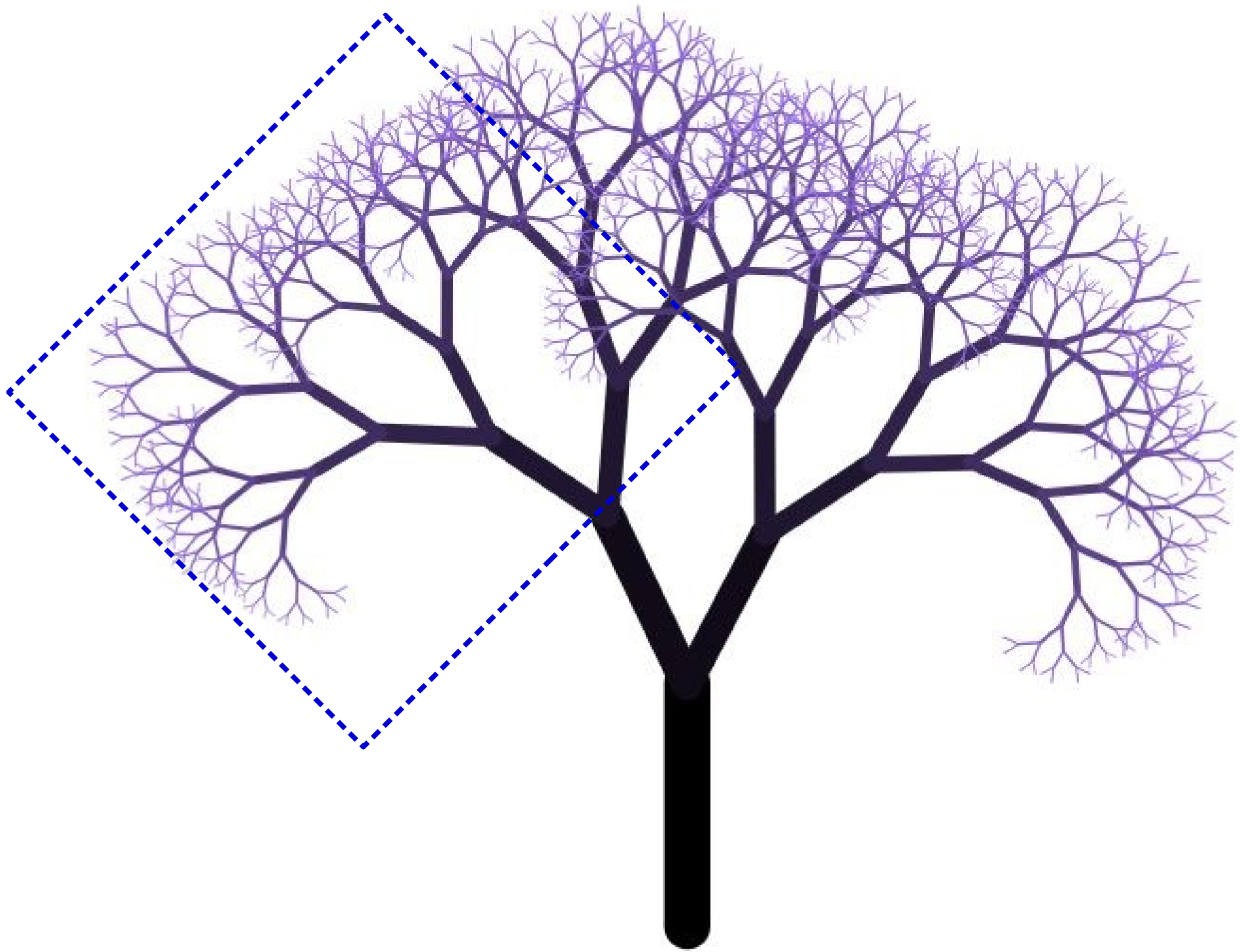
Quantitatively analyze different approaches for solving problems.



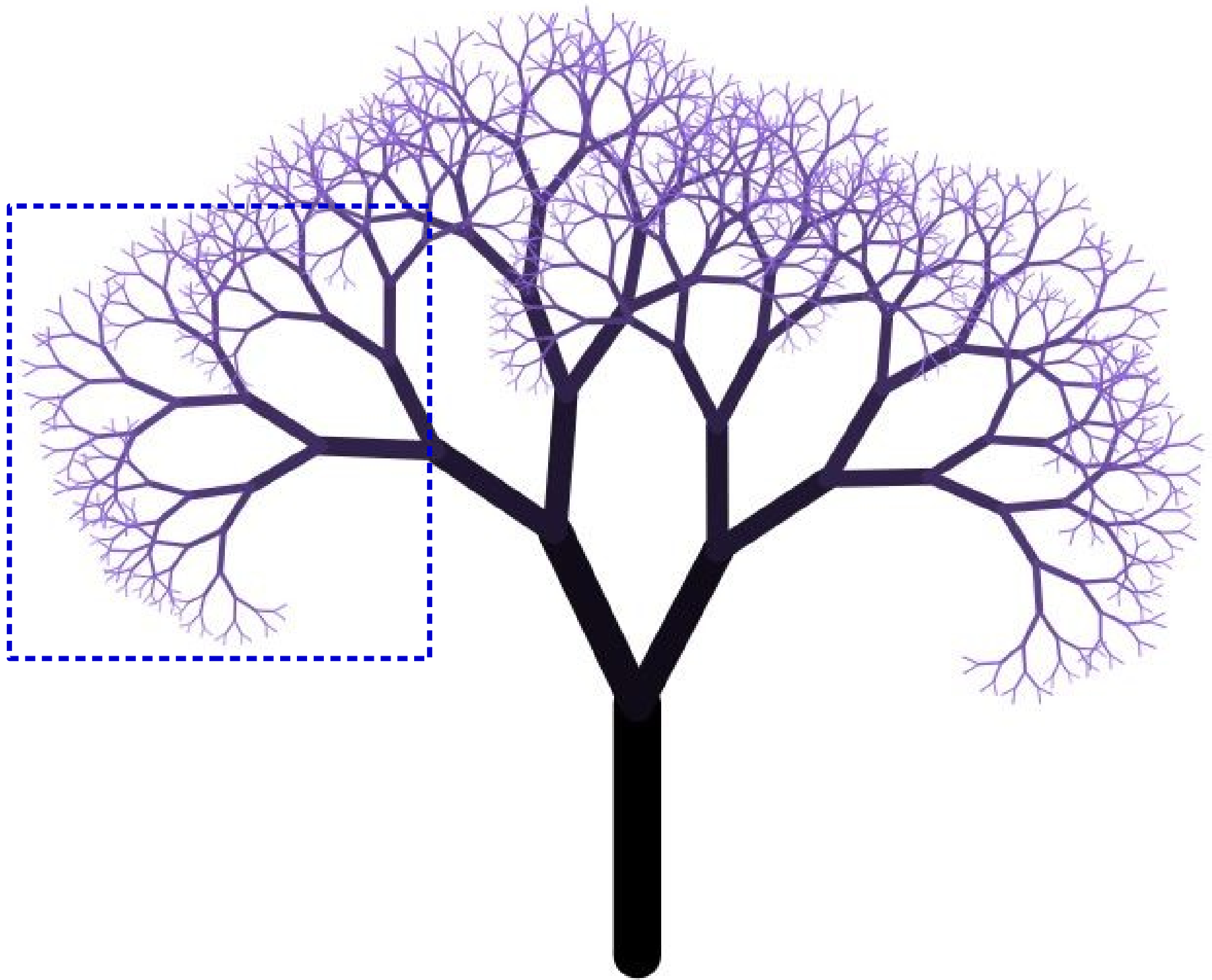
<http://www.marketoracle.co.uk/images/2010/Oct/fractal-tree2.jpg>



<http://www.marketoracle.co.uk/images/2010/Oct/fractal-tree2.jpg>



<http://www.marketoracle.co.uk/images/2010/Oct/fractal-tree2.jpg>



<http://www.marketoracle.co.uk/images/2010/Oct/fractal-tree2.jpg>

A ***recursive solution*** is a solution that is defined in terms of itself.

Goals for this Course

- ***Learn how to model and solve complex problems with computers.***
- To that end:
 - Explore common abstractions for representing problems.
 - Harness recursion and understand how to think about problems recursively.
 - Quantitatively analyze different approaches for solving problems.

Goals for this Course

Learn how to model and solve complex problems with computers.

To that end:

Explore common abstractions for representing problems.

Harness recursion and understand how to think about problems recursively.

- Quantitatively analyze different approaches for solving problems.

ull,"status":"reviewed","tsunami":0,"sig":369,"net":"us","code":"2000j048","ids":"","us2000j048",
origin,phase-data,"","nst":null,"dmin":1.598,"rms":0.78,"gap":104,"magType":"mww","type":"earthquake",
Tobelo, Indonesia"},"geometry":{"type":"Point","coordinates":[127.3157,2.3801,53.72]},"id":"us2000j048",
{"type":"Feature","properties":{"mag":5.1,"place":"265km SW of Severo-Kuril'sk, Russia","time":1546548377590,"updated":1546549398040,"tz":600,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j03t.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":400,"net":"us","code":"2000j03t","ids":"","us2000j03t"},
gin,phase-data,"","nst":null,"dmin":5.198,"rms":0.94,"gap":48,"magType":"mww","type":"earthquake",
Severo-Kuril'sk, Russia"},"geometry":{"type":"Point","coordinates":[153.7105,48.8712,104.7]},"id":"us2000j03t",
{"type":"Feature","properties":{"mag":4.8,"place":"20km NNW of Taitung City, Taiwan","time":1546538570070,"updated":1546541624040,"tz":480,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j02k.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":354,"net":"us","code":"2000j02k","ids":"","us2000j02k"},
gin,phase-data,"","nst":null,"dmin":0.52,"rms":0.79,"gap":110,"magType":"mb","type":"earthquake",
City, Taiwan"},"geometry":{"type":"Point","coordinates":[121.0489,22.9222,10]},"id":"us2000j02k",
{"type":"Feature","properties":{"mag":5,"place":"79km ENE of Petropavlovsk-Kamchatskiy, Russia","time":1546538266300,"updated":1546541474965,"tz":720,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j02g.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":385,"net":"us","code":"2000j02g","ids":"","us2000j02g"},
us":"reviewed","tsunami":0,"sig":385,"net":"us","code":"2000j02g","ids":"","us2000j02g","so",
in,phase-data,"","nst":null,"dmin":0.728,"rms":0.75,"gap":114,"magType":"mb","type":"earthquake",
Petropavlovsk-Kamchatskiy, Russia"},"geometry":{"type":"Point","coordinates":[159.6844,53.7]},"id":"us2000j02g",
{"type":"Feature","properties":{"mag":4.5,"place":"South of Java, Indonesia","time":1546533739000,"updated":1546539809085,"tz":420,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j024.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":312,"net":"us","code":"2000j024","ids":"","us2000j024"},
,"detail":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j024.geojson","felt":0,"status":"reviewed",
tatus":"reviewed","tsunami":0,"sig":312,"net":"us","code":"2000j024","ids":"","us2000j024","so",
igin,phase-data,"","nst":null,"dmin":2.821,"rms":0.89,"gap":83,"magType":"mb","type":"earthquake",
Indonesia"},"geometry":{"type":"Point","coordinates":[108.5165,-10.6419,8.84]},"id":"us2000j024",
{"type":"Feature","properties":{"mag":4.8,"place":"108km N of Ishigaki, Japan","time":1546529675300,"updated":1546530815040,"tz":480,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j01x.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":354,"net":"us","code":"2000j01x","ids":"","us2000j01x"},
etail":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j01x.geojson","felt":0,"status":"reviewed",
"status":"reviewed","tsunami":0,"sig":354,"net":"us","code":"2000j01x","ids":"","us2000j01x","so",
in,phase-data,"","nst":null,"dmin":1.342,"rms":0.82,"gap":68,"magType":"mb","type":"earthquake",
Japan"},"geometry":{"type":"Point","coordinates":[124.1559,25.3209,122.33]},"id":"us2000j01x",
{"type":"Feature","properties":{"mag":5.4,"place":"82km S of Bristol Island, South Sandwich Islands","time":1546519662810,"updated":1546520523040,"tz":-120,"url":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j01b.geojson","felt":0,"status":"reviewed","tsunami":0,"sig":400,"net":"us","code":"2000j01b","ids":"","us2000j01b"},
"detail":"https://earthquake.usgs.gov/earthquakes/feed/v1.0/detail/us2000j01b.geojson","felt":0,"status":"reviewed",

There are many ways to solve the same problem. How do we ***quantitatively*** talk about how they compare?

Goals for this Course

- ***Learn how to model and solve complex problems with computers.***
- To that end:
 - Explore common abstractions for representing problems.
 - Harness recursion and understand how to think about problems recursively.
 - Quantitatively analyze different approaches for solving problems.

Who's Here Today?

- Aeronautics and Astronautics
- Bioengineering
- Biology
- Chemical Engineering
- Chemistry
- Civil / Environmental Engineering
- Computational and Mathematical Engineering
- Computer Science
- Creative Writing
- Design
- Earth Planetary Sciences
- Earth Systems
- Economics
- Education
- Electrical Engineering
- Engineering
- Ethics in Society
- History
- Human Biology
- Immunology
- Law
- Mechanical Engineering
- Medicine
- Management Science and Engineering
- Physics
- Political Science
- Spanish
- Statistics
- Stem Cell Engineering / Regenerative Medicine
- Symbolic Systems
- Theater and Performance Studies
- ***Undeclared!***

Transitioning to C++

Transitioning to C++

- I'm assuming that the majority of you are either coming out of CS101A in Python coming from AP CS in Java.
- In this course, we'll use the C++ programming language.
- Learning a second programming language is ***substantially*** easier than learning a first.
 - You already know how to solve problems; you just need to adjust the syntax you use.
- While the languages are superficially different, they have much in common.

Our First C++ Program

Prime Numbers

- A positive integer n is called a ***prime number*** if it's greater than one and its only positive divisors are 1 and n .
- For example:
 - 15 isn't prime ($15 = 3 \times 5$).
 - 17 is prime.
 - 19 is prime.
 - 21 isn't prime ($21 = 3 \times 7$).
- What's the 5,000th prime number?

```
def isPrime(n):  
    """Returns whether the input number is prime; assumes n >= 2."""  
  
    # Try dividing by all numbers from 2 to n - 1, inclusive.  
    for divisor in range(2, n):  
        if n % divisor == 0:  
            return False  
  
    return True
```

```
found = 0    # How many prime numbers we've found  
number = 1   # Next number to test  
  
# Keep trying numbers until we've found 5000 primes.  
while found < 5000:  
    number += 1  
    if isPrime(number):  
        found += 1  
  
# Last number tried is the 5000th prime.  
print(number)
```

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;
        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

In Python, indentation alone determines nesting.

In C++, indentation is nice, but **curly braces** alone determine nesting.


```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

```

Python uses **True** and **False**;
C++ uses **true** and **false**.

```

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n-1. */
    for (int divisor = 2; divisor < n; divisor++)
        if (n % divisor == 0) {
            return false;
        }
    return true;
}

int main() {
    int found = 0; // How many primes found
    int number = 1; // Number to test
    /* Keep trying numbers until we find 5000 primes. */
    while (found < 5000)
        number++;
        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

In Python, newlines mark the end of statements.

In C++, individual statements must have a semicolon (;) after them.

return false;

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;
        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the next prime. */
    cout << number << endl;
    return 0;
}

```

In Python, you print output by using `print()`.

In C++, you use the ***stream insertion operator*** (`<<`) to push data to the console. (Pushing `endl` prints a newline.)

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n-1 */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

In Python, you can optionally put parentheses around conditions in if statements and while loops.

In C++, these are mandatory.

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0)
            return false;
    }
    return true;
}

```

Python and C++ each have **for** loops, but the syntax is different. (Check the textbook for more details about how this works!)

```

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++) {
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int found = 0; // How many primes found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;
        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

C++ has an operator ++ that means “change this variable’s value by adding one to it.” Python doesn’t have this.

```
#include <iostream>
using namespace std;
```

```
/* Returns whether the input number is prime; assumes n >= 2. */
```

```
bool isPrime(int n) {
```

```
/* Try dividing by all numbers from 2 to n - 1, inclusive. */
```

```
for (int divisor = 2; divisor < n; divisor++) {
```

```
    if (n % divisor == 0)
```

```
        return false;
```

```
}
```

```
}
```

```
return true;
```

```
}
```

```
int main() {
```

```
    int found = 0; // How many prime numbers we've found
```

```
    int number = 1; // Next number to test
```

```
/* Keep trying numbers until we've found 5000 primes. */
```

```
while (found < 5000) {
```

```
    number++;
```

```
    if (isPrime(number)) {
```

```
        found++;
```

```
    }
```

```
}
```

```
/* Last number tried is the 5000th prime. */
```

```
cout << number << endl;
```

```
return 0;
```

```
}
```

In Python, comments start with # and continue to the end of the line.

In C++, there are two styles of comments. Comments starting with /* continue until */. Comments starting with // continue to the end of the line.

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1, inclusive. */
    for (int divisor = 2; divisor < n; divisor++)
        if (n % divisor == 0)
            return false;

    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number))
            found++;
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

In Python, comments on functions are customarily written after the first line of the function.

In C++, comments on functions are customarily written *before* the first line of the function.


```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n - 1 inclusive. */
    for (int divisor = 2; divisor < n; divisor++)
        if (n % divisor == 0) {
            return false;
        }
    return true;
}

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

In Python, each object has a type, but it isn't stated explicitly.

In C++, you *must* give a type to each variable. (The **int** type represents an integer, and **bool** represents a Boolean true or false.)

```

#include <iostream>
using namespace std;

/* Returns whether the input number is prime; assumes n >= 2. */
bool isPrime(int n) {
    /* Try dividing by all numbers from 2 to n-1. */
    for (int divisor = 2; divisor < n; divisor++)
        if (n % divisor == 0) {
            return false;
        }
    return true;
}

```

In Python, statements can be either in a function or at the top level of the program.

In C++, most statements must be inside of a function.

```

int main() {
    int found = 0; // How many prime numbers we've found
    int number = 1; // Next number to test

    /* Keep trying numbers until we've found 5000 primes. */
    while (found < 5000) {
        number++;

        if (isPrime(number)) {
            found++;
        }
    }

    /* Last number tried is the 5000th prime. */
    cout << number << endl;
    return 0;
}

```

Why do we have both C++ and Python?

C++ and Python

- Python is a *great* language for data processing and writing quick scripts across all disciplines.
 - It's pretty quick to make changes to Python programs and then run them to see what's different.
 - Python programs, generally, run more slowly than C++ programs.
- C++ is a *great* language for writing high-performance code that takes advantage of underlying hardware.
 - Compiling C++ code introduces some delays between changing the code and running the code.
 - C++ programs, generally, run much faster than Python programs.
- Knowing both languages helps you use the right tool for the right job.

Your Action Items

- ***Read Chapter 1 of the textbook.***
 - Use this as an opportunity to get comfortable with the basics of C++ programming and to read more examples of C++ code.
- ***Start Assignment 0.***
 - Assignment 0 is due this Friday half an hour before the start of class (1:00PM Pacific time). The assignment and its starter files are up on the course website.
 - No programming involved, but you'll need to get your development environment set up.
 - There's a bunch of documentation up on the course website. Please feel free to reach out to us if there's anything we can do to help out!

Next Time

- ***Welcome to C++!***
 - Defining functions.
 - Basic arithmetic.
 - Writing loops.